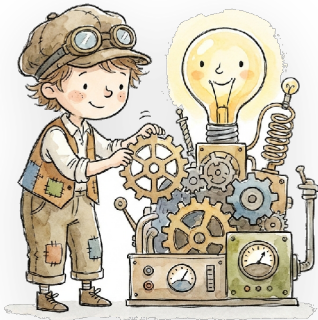




## PROBABILITY &amp; STATISTICS

# Evaluating a Model: Accuracy, Precision, Recall



## YOUR GUIDE

*Let's get started.*

We can start with a number that should make us suspicious: **99% accurate**. For example, suppose we have a model that flags fraudulent credit-card transactions, and it is right 99% of the time. This sounds like a triumph; however, we must notice that only 1 in 100 transactions is actually fraud. Consequently, a model that simply answers "not fraud" every single time, and that has learned *nothing*, scores exactly 99% too. It catches zero fraud and still looks excellent on paper. The gap between a

model that looks good and a model that *is* good is the main subject of this chapter. We can begin closing this gap with one small grid of four numbers.

In [the naive-bayes classifier](#), we built a model that outputs a class. This chapter is about the harder, more rigorous question that comes right after: how good is the model, *really*? We can build the answer up from a concrete example, because the abstractions only become clear once we have counted the cases by hand.

## The four outcomes: a confusion matrix

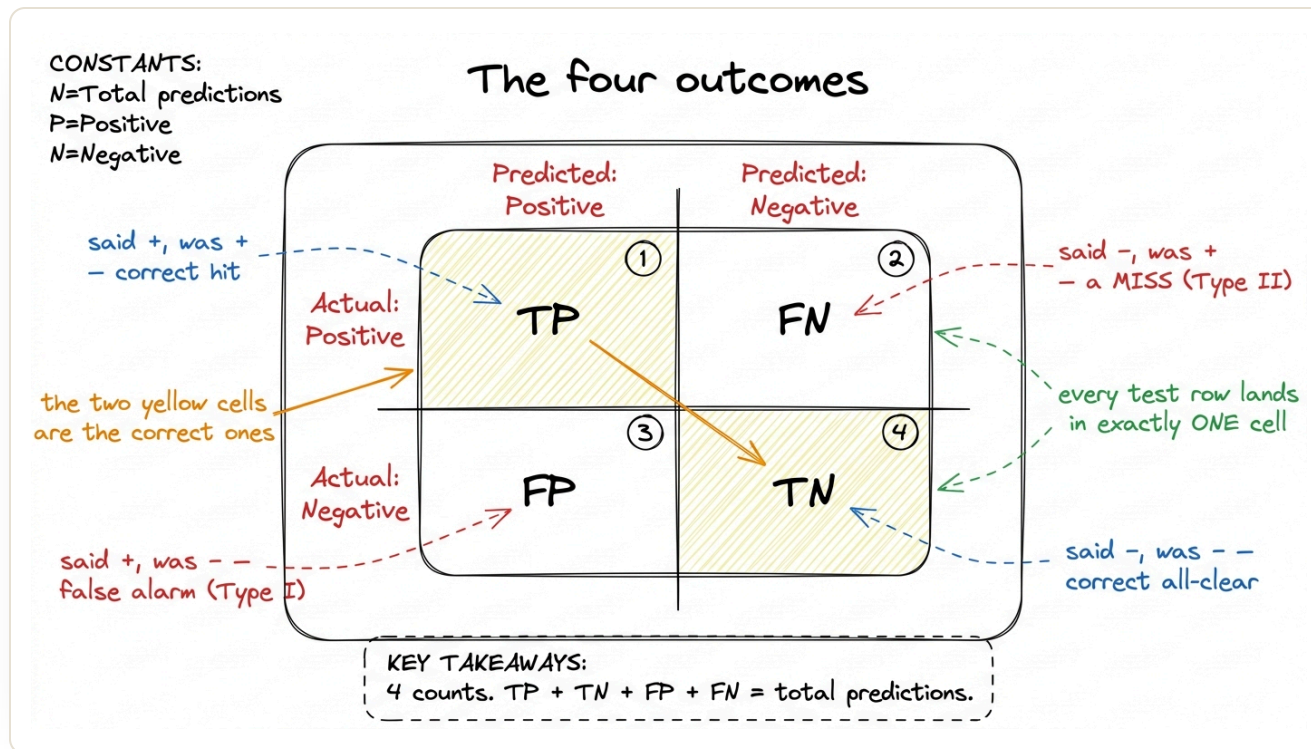
---

For example, we can consider a binary classifier, which is one that says **positive** or **negative**. This includes tasks like predicting cancer or no cancer, spam or not spam, and fraud or legit. For every prediction it makes, there are exactly four things that can happen, and they come from crossing what the model *said* against what was *actually true*.

- **True Positive (TP)**: it said positive, and it was positive. A correct alarm.

- **True Negative (TN)**: it said negative, and it was negative. A correct all-clear.
- **False Positive (FP)**: it said positive, but it was actually negative. A false alarm. (Statisticians call this a Type I error.)
- **False Negative (FN)**: it said negative, but it was actually positive. A miss. (A Type II error.)<sup>1</sup>

Arrange those four counts in a 2×2 grid and you have the **confusion matrix** — actual class along one axis, predicted class along the other. Every single row of your test set lands in exactly one of the four cells. Nothing hides.



*The confusion matrix crosses what the model predicted against what was actually true; the two shaded diagonal cells are the correct predictions.*

We can illustrate this concept with a tiny, countable example. Suppose we run a cancer-screening model on **100 patients**. In truth, **10 of them have cancer** and 90 do not. As the output shows, the model produces this confusion matrix:

- **TP = 8** — it caught 8 of the 10 real cancers
- **FN = 2** — it missed 2 real cancers, telling those patients they were fine

- $FP = 5$  — it alarmed on 5 healthy patients
- $TN = 85$  — it correctly cleared 85 healthy patients

We can check the arithmetic:  $8 + 2 + 5 + 85 = 100$ . Note that every patient is accounted for. Now we can compute every metric that matters by reading the numbers off this grid.

## Accuracy, and why it lies

---

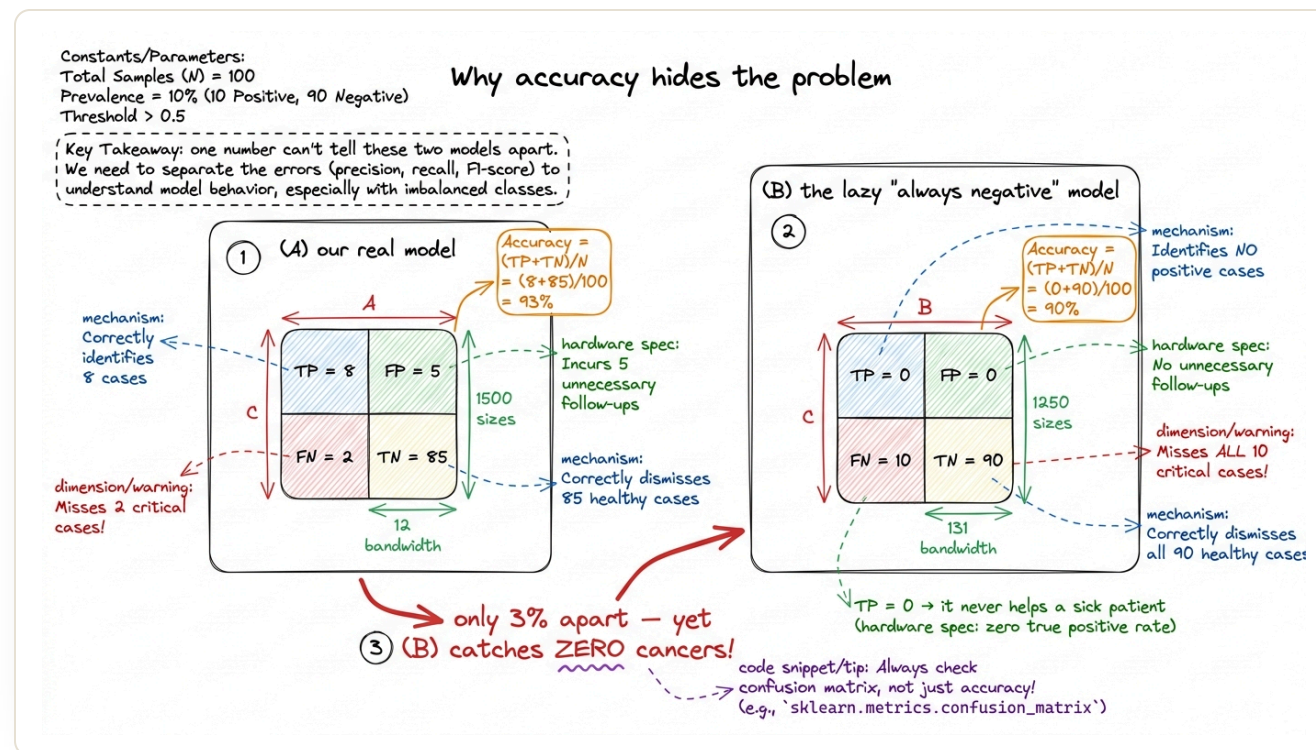
**Accuracy** is the obvious first metric: of all predictions, what fraction did we get right? That is the two correct cells over the total.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{8 + 85}{100} = \frac{93}{100} = 93\%$$

The result is 93% correct. This seems reasonable; however, we must remember the fraud example from the opening. Accuracy has a fatal blind spot, because it treats every error the same and gets dominated by whichever class is bigger. On our cancer set, a lazy model that predicts "no cancer" for *everyone* would score  $TN = 90$ ,  $TP = 0$ , for an accuracy of 90%. This is barely worse than our real model; however, it catches not a single

cancer.<sup>2</sup> Note that when the classes are imbalanced, accuracy flatters the model that plays it safe.

So the key move is this: stop measuring "how often are you right?" and start measuring the two different ways of being wrong separately. That is what precision and recall do.



*A do-nothing model scores nearly the same accuracy as a real one on imbalanced data — proof that a single accuracy number can be dangerously misleading.*



## Precision: when you sound the alarm, are you right?

---

**Precision** asks a focused question: *of everything the model flagged as positive, what fraction really was positive?* This metric lives entirely in the "Predicted: Positive" column. For example, we compute it as the true hits divided by all the alarms.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{8}{8 + 5} = \frac{8}{13} \approx 0.615 = 61.5\%$$

When our model predicts "cancer!", it is correct about 62% of the time, and the other 38% are false alarms sent to healthy people. Precision is the metric we care about when a **false positive is expensive**. For example, this applies to a spam filter that dumps a real job offer into junk, a fraud system that freezes a legitimate customer's card at a checkout, or a court risk-score that flags an innocent person. High precision means that *when the model makes a positive prediction, we can trust it*.

## Recall: of everything you should have caught, how much did you catch?

---

**Recall** (also called *sensitivity* or the *true positive rate*) addresses the mirror-image question: *of all the truly positive cases out there, what fraction did the model actually catch?* We compute this metric entirely within the "Actual: Positive" row. Specifically, it is the true hits divided by all the real positives.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{8}{8 + 2} = \frac{8}{10} = 0.8 = 80\%$$

Our model caught 80% of the real cancers and missed 20% of them. Recall is what you care about when a **false negative is catastrophic** — a missed tumor, an undetected fraud, a security threat waved through. High recall means: *it rarely lets a real one slip past.*



## Precision vs Recall – same TP, different denominators

**PRECISION**

|                 | Predicted Positive | Predicted Negative |
|-----------------|--------------------|--------------------|
| Actual Positive | TP = 8             | FN = 2             |
| Actual Negative | FP = 5             | TN = 15            |

Precision =  $\frac{TP}{TP+FP} = \frac{8}{13}$

reads DOWN the column – of my alarms, how many were real?

**RECALL**

|                 | Predicted Positive | Predicted Negative |
|-----------------|--------------------|--------------------|
| Actual Positive | TP = 8             | FN = 2             |
| Actual Negative | FP = 5             | TN = 15            |

Recall =  $\frac{TP}{TP+FN} = \frac{8}{10}$

reads ACROSS the row – of the real cases, how many did I catch?

same TP on top – different denominator on the bottom

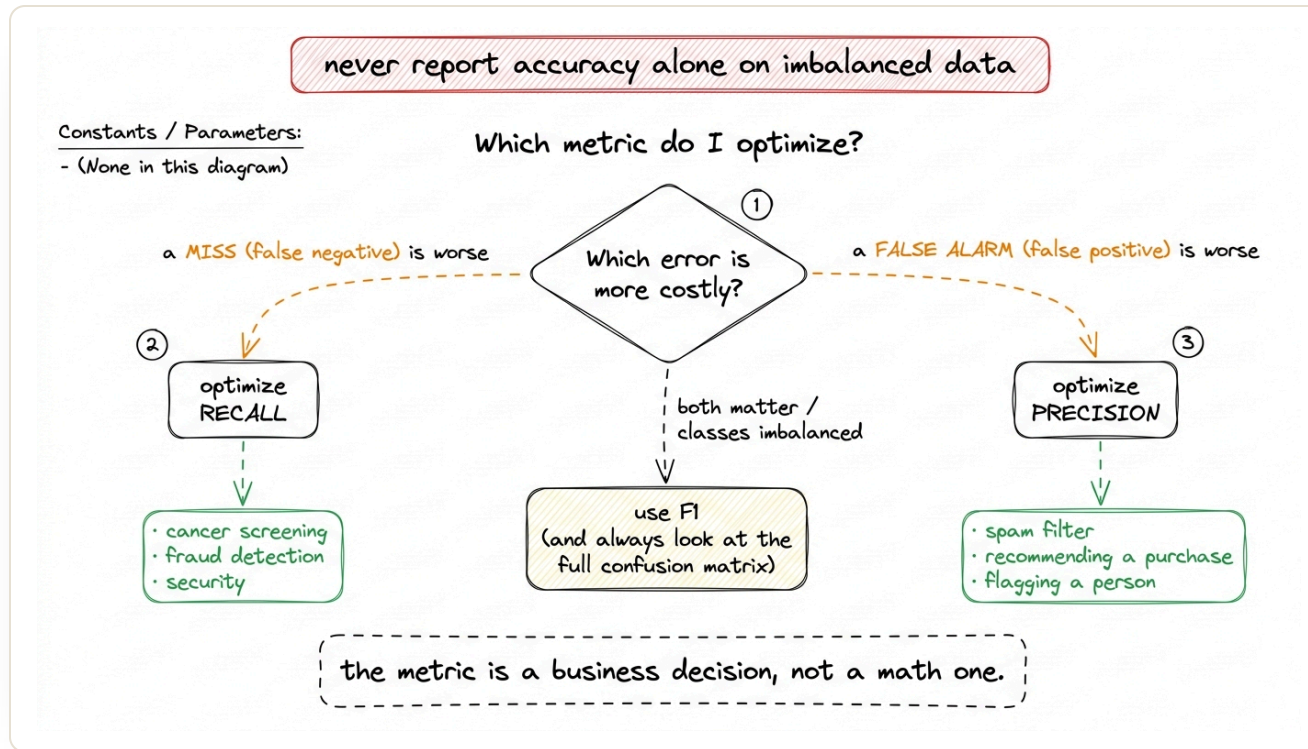
Precision guards against false alarms; Recall guards against misses.

*Precision divides the true hits by the whole predicted-positive column; recall divides the same hits by the whole actual-positive row. Same numerator, different question.*

Note that there is a tension here. We could make recall a perfect 100% in one line by simply predicting "cancer" for every patient. Then  $FN = 0$ , we miss nobody, however, we have also alarmed all 90 healthy people, so precision collapses. If we configure the model to predict positive cases more often, recall rises while precision falls. Conversely, if we make it cautious, precision rises while recall falls. This is the famous **precision–recall trade-off**, and it is governed by the model's decision **threshold**. This

threshold is the probability cutoff above which we call something positive.<sup>3</sup>

It helps to observe this process with real numbers. Suppose our model outputs a probability per patient and we sweep the threshold from strict to lenient. At a **high threshold of 0.9**, the model only predicts "cancer" when it is very sure, so it flags just 6 patients: 5 true cancers and 1 false alarm. That gives a precision of  $5/6 = 0.83$  but a recall of only  $5/10 = 0.50$ , since it stayed quiet on 5 real cancers. If we drop the threshold to 0.5, it becomes less strict, flagging 13 patients (8 real, 5 false), which is the exact grid we computed above. Here, precision falls to  $8/13 = 0.62$ , and recall climbs to  $8/10 = 0.80$ . If we drop it again to a lenient 0.2, it flags 30 patients, catching all 10 cancers but with 20 false alarms. As a result, recall is now a perfect  $1.0$ , however, precision has dropped to  $10/30 = 0.33$ . Note that nothing about the model changed between these three rows. The only thing that moved was one parameter, and it slid the two metrics in opposite directions every time. This is why a single precision or recall number is meaningless without stating the threshold it was measured at.



*Sliding the decision threshold from strict to lenient trades precision for recall along the same model, which is why a metric is only meaningful alongside the threshold it was measured at.*

## F1: one number that refuses to be fooled

Sometimes we need a single score, for example, to rank models or to plot a training curve. Accuracy was disqualified earlier. The natural repair is to combine precision and recall, and we can combine them with a **harmonic mean** rather than a plain average. This calculation gives the **F1 score**:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

We can plug in our numbers, specifically precision **0.615** and recall **0.8**:

$$F_1 = 2 \cdot \frac{0.615 \times 0.8}{0.615 + 0.8} = 2 \cdot \frac{0.492}{1.415} \approx 0.696 = 69.6\%$$

We might ask why we use the harmonic mean and not the ordinary average. This is because the harmonic mean punishes imbalance. For example, consider the "predict cancer for everyone" model, which has a recall of **1.0**, but a precision of only  $10/100 = 0.1$ . Its plain average is a relatively high **0.55**, however, its F1 score is  $2 \cdot \frac{0.1 \times 1.0}{0.1 + 1.0} \approx 0.18$ .<sup>4</sup> The harmonic mean accounted for the imbalance that the plain average missed. **Note that the F1 score is only high when precision and recall are *both* high.**

## Reading it all off in code

---

In practice you never compute these by hand past the first few times — **scikit-learn** reads them straight off predictions and true labels. Here is the whole thing on our toy set, reconstructed to match the confusion matrix above:

```
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1

# 1 = cancer (positive), 0 = no cancer. 100 patients.
y_true = [1]*10 + [0]*90          # 10 real cancers, 90 healthy
y_pred = [1]*8 + [0]*2 + [1]*5 + [0]*85  # 8 TP, 2 FN, 5 FP, 85 TN

print(confusion_matrix(y_true, y_pred))    # [[85  5]
                                           #   [ 2  8]]

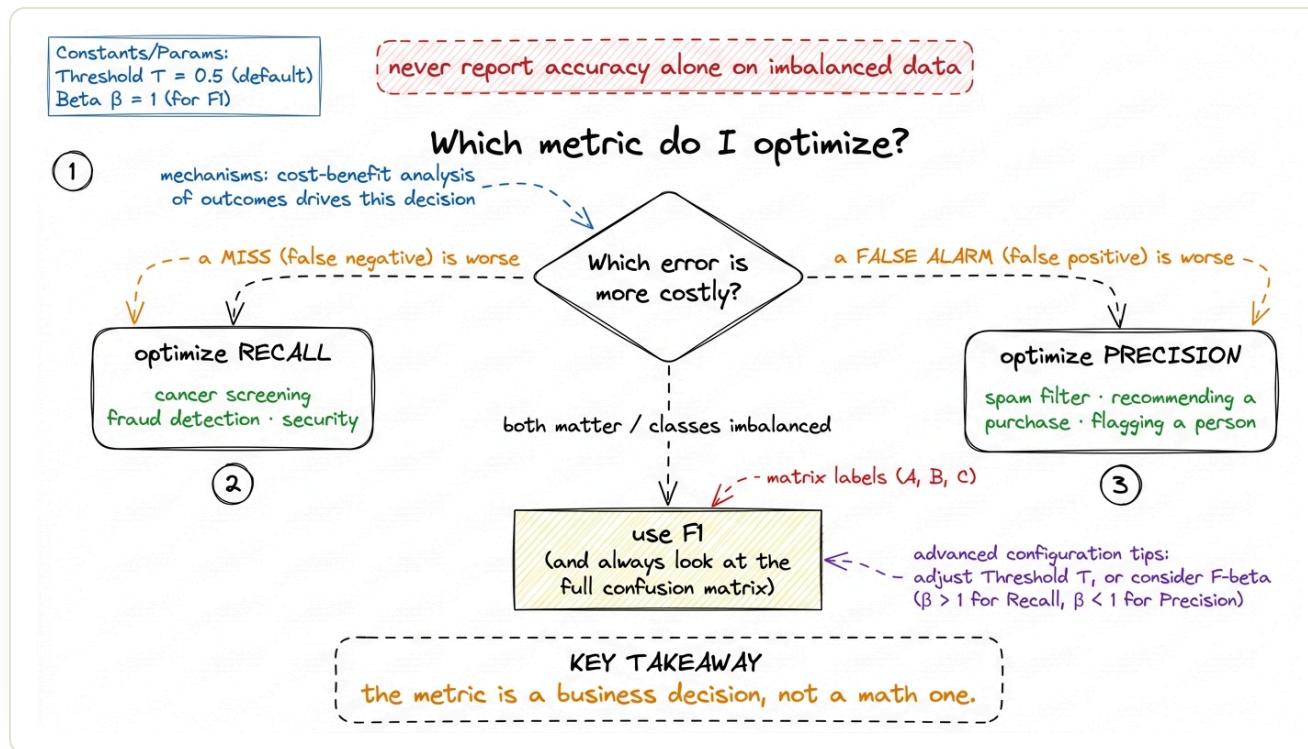
print(precision_score(y_true, y_pred))    # 0.615...
print(recall_score(y_true, y_pred))       # 0.8
print(f1_score(y_true, y_pred))           # 0.696...
```

One caution that trips everyone up: `confusion_matrix` orders its axes by sorted label, so with labels `0` and `1` it puts class `0` first — the top-left cell is `TN`, not `TP`, the reverse of how we drew it above.<sup>5</sup>

## The one decision that picks your metric

Strip away the formulas and evaluation comes down to a single question you must answer *before* you look at any number: **which mistake hurts more, a false alarm or a miss?**





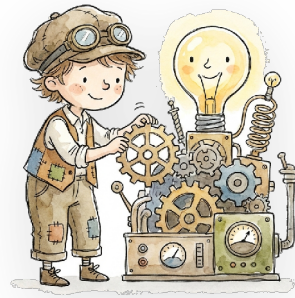
A cheat-sheet: recall when a miss is worse, precision when a false alarm is worse, F1 when both matter  
— and never accuracy alone on imbalanced data.

Note that there is no universally "best" metric, only the metric that matches the cost of being wrong in *your* problem. For example, a cancer screen prizes recall and tolerates false alarms, because a follow-up test is cheap and a missed tumor is not. A spam filter, however, prizes precision, because losing a real email is worse than seeing one junk message. We should choose the metric first and then evaluate it alongside the full

confusion matrix. By doing this, we will never again be fooled by a seemingly impressive 99%.

We now have a trustworthy way to *score* a model. The next logical question is how a model *improves* its score. However, to achieve this, the model must nudge its parameters to make fewer mistakes. That underlying machinery runs on slopes and rates of change, which is exactly the topic we will explore when differential calculus: derivatives and gradients picks up the thread.

---



CHAPTER COMPLETE

*Nicely done. You reached the end of this one.*



[NEXT CHAPTER](#) **Differential Calculus: Derivatives and Gradients****0**

Did this chapter land for you?

**Mark chapter as done**

Rate this chapter ★ ★ ★ ★ ★

**What did you think of this chapter?**

A takeaway, a question, a moment that clicked...



Be kind and specific. 0/2000

**Post***No notes yet — be the first to share what you took away.*[◀ The Naive-Bayes Classif...](#)[◊ Back to book](#)[Differential Calculus: ... >](#)